

Advanced Techniques (3): Hyperparameter Tuning with Optuna

Tutorial 1 compared feature sets and models; Tutorial 2 added domain features. In this notebook we hold the feature set fixed and change *how the model learns*.

Tuning is different from feature engineering: the input columns stay fixed, and we adjust the model's settings. The aim is not to trust an automatic search blindly, but to run it as a controlled experiment — score a default model first, search, and then check whether the result actually generalizes. A tuned model is a claim worth submitting only if cross-validation says it beats the untuned one.

Task: Predict the probability that an NFL prospect is **Drafted**. **1** means the player was drafted, and **0** means the player was not drafted.

Evaluation metric: ROC-AUC.

Connection to Previous Lectures

In this notebook, we apply several ideas from the previous lectures.

- From **Supervised Learning**, we use a classification model to predict **Drafted**.
- From **Model Evaluation**, we compare models using ROC-AUC.
- From **Cross-Validation**, we use the same validation strategy when comparing default and tuned models.
- From **Feature Engineering**, we reuse the feature set created in Tutorials 1 and 2.
- From **Hyperparameter Search**, we treat model settings as something to evaluate with validation, not something to choose by intuition alone.

Contents

1. Setup
2. Load the data
3. Understand the data
4. Preprocessing and feature set
5. CatBoost baseline
6. Hyperparameter tuning with Optuna
7. Refit the tuned model
8. Create the submission file
9. Wrap-up

1. Setup

1.1 Import libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from pathlib import Path

import warnings
warnings.filterwarnings('ignore')

SEED = 42
np.random.seed(SEED)
```

```
# Install required libraries if they are not available in your environment.
!pip install -q optuna catboost
```

1.2 Connect with Google Drive

```
# If you are using Google Colaboratory, run this cell as well.
from google.colab import drive
```

```
drive.mount('/content/drive')
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

Next, move to the folder that contains this notebook and the `input/` folder.

```
# Specify the directory where this notebook is located after %cd.
%cd "/content/drive/MyDrive/competition"
```

```
/content/drive/MyDrive/competition
```

Run the cell below to check the path is correct.

```
from pathlib import Path

WORK_DIR = Path.cwd()
PATH = WORK_DIR / 'input'
train_file = PATH / 'train.csv'
test_file = PATH / 'test.csv'
sample_sub_file = PATH / 'sample_submission.csv'

if train_file.exists() and test_file.exists() and sample_sub_file.exists():
    print('All required files exist and the path is set correctly.')
else:
    print('Some files are missing or the path is not set correctly.')
```

All required files exist and the path is set correctly.

2. Load the data

IMPORTANT: Whenever you change a feature, a preprocessing step, or a model, re-run **every cell from this data-loading cell**

downward so that `train_raw` and `test_raw` are rebuilt from the original files. Many strange errors come from a stale DataFrame left over from a previous run.

```
train = pd.read_csv(train_file)
test = pd.read_csv(test_file)
sample_sub = pd.read_csv(sample_sub_file)

# We keep raw copies so that preprocessing can always start from the same data.
train_raw = train.copy()
test_raw = test.copy()

print(f'train: {train.shape}, test: {test.shape}, sample_sub: {sample_sub.shape}')
```

train: (2781, 16), test: (696, 15), sample_sub: (696, 2)

```
train.head()
```

	Id	Year	Age	School	Height	Weight	Sprint_40yd	Vertical_Jump	Bench_Press_Reps	Broad_Jump	Agility_3cone	Shuttle
0	0	2011	21.0	Lehigh	1.9050	140.160042	5.39	59.69	29.0	251.46	7.91	4.94
1	1	2011	24.0	Abilene Christian	1.8288	87.089735	4.31	101.60	16.0	332.74	NaN	NaN
2	2	2018	21.0	Colorado St.	1.8542	92.986436	4.51	91.44	10.0	309.88	6.95	4.37
3	3	2010	21.0	East	1.9304	148.778207	5.09	76.20	39.0	254.00	8.12	4.71

Next steps: [Generate code with train](#) [New interactive sheet](#)

```
train.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2781 entries, 0 to 2780
Data columns (total 16 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Id                    2781 non-null  int64
```

```

1 Year          2781 non-null int64
2 Age           2346 non-null float64
3 School        2781 non-null object
4 Height        2781 non-null float64
5 Weight        2781 non-null float64
6 Sprint_40yd   2636 non-null float64
7 Vertical_Jump 2227 non-null float64
8 Bench_Press_Reps 2060 non-null float64
9 Broad_Jump    2200 non-null float64
10 Agility_3cone 1811 non-null float64
11 Shuttle      1869 non-null float64
12 Player_Type  2781 non-null object
13 Position_Type 2781 non-null object
14 Position     2781 non-null object
15 Drafted      2781 non-null float64
dtypes: float64(10), int64(2), object(4)
memory usage: 347.8+ KB

```

3. Understand the data

Tutorials 1 and 2 covered the EDA; here we only confirm the data is ready to tune — the target is imbalanced, the combine columns still have gaps, and the preprocessing matches train and test. Tuning is worth doing only once the feature set and validation are stable.

3.1 Missing values

```

missing_summary = (
    train_raw
    .isna()
    .sum()
    .to_frame('missing_count')
)

missing_summary['missing_rate'] = missing_summary['missing_count'] / len(train_raw)
missing_summary.sort_values('missing_rate', ascending=False)

```

	missing_count	missing_rate	
Agility_3cone	970	0.348795	
Shuttle	912	0.327940	
Bench_Press_Reps	721	0.259259	
Broad_Jump	581	0.208918	
Vertical_Jump	554	0.199209	
Age	435	0.156419	
Sprint_40yd	145	0.052140	
Id	0	0.000000	
Height	0	0.000000	
Weight	0	0.000000	
Year	0	0.000000	
School	0	0.000000	
Player_Type	0	0.000000	
Position_Type	0	0.000000	
Position	0	0.000000	
Drafted	0	0.000000	

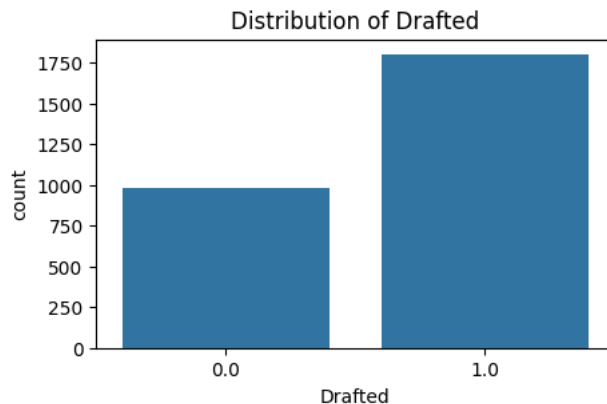
Findings:

- The preprocessing is the same as the earlier tutorials, so the feature set is held fixed here.
- Missing values are handled before imputation, keeping the "did not test" signal.

3.2 Target balance

```
plt.figure(figsize=(5, 3))
sns.countplot(data=train_raw, x='Drafted')
plt.title('Distribution of Drafted')
plt.show()

(train_raw['Drafted'].value_counts(normalize=True) * 100).round(2)
```



```
proportion
Drafted
1.0    64.83
0.0    35.17
```

dtype: float64

Findings:

- The target is imbalanced, and the competition is scored on **ROC-AUC**.

3.3 Combine measurements

```
combine_cols = [
    'Height', 'Weight', 'Sprint_40yd', 'Vertical_Jump',
    'Bench_Press_Reps', 'Broad_Jump', 'Agility_3cone', 'Shuttle'
]

train_raw[combine_cols].describe().T
```

	count	mean	std	min	25%	50%	75%	max
Height	2781.0	1.873590	0.067178	1.651000	1.828800	1.879600	1.930400	2.082800
Weight	2781.0	109.502026	20.307239	67.585263	92.986436	104.779837	124.284309	162.386068
Sprint_40yd	2636.0	4.764818	0.299343	4.220000	4.530000	4.680000	4.950000	6.000000
Vertical_Jump	2227.0	83.506349	10.710691	44.450000	76.200000	83.820000	91.440000	114.300000
Bench_Press_Reps	2060.0	20.236408	6.410662	2.000000	16.000000	20.000000	24.000000	45.000000
Broad_Jump	2200.0	291.985700	23.924666	198.120000	276.860000	294.640000	307.340000	373.380000
Agility_3cone	1811.0	7.230447	0.410686	6.280000	6.930000	7.130000	7.490000	9.040000
Shuttle	1869.0	4.399422	0.263851	3.820000	4.200000	4.350000	4.560000	5.560000

Findings:

- These drills are the raw material for the athletic features built in the previous tutorial, which we now hold fixed.

Questions to think about:

- With the features frozen, what is the only thing left that can move the score in this notebook?
- With only a few thousand rows, how much fold-to-fold swing in AUC would you treat as noise?

4. Preprocessing and feature set

We reuse the exact feature set from Tutorials 1 and 2 — the simple features, the domain features, and the same imputation and encoding. Holding the features fixed means any score change in this notebook comes from the model settings alone.

Questions to think about:

- We freeze the features to isolate the model. Why is changing features *and* hyperparameters at the same time so hard to interpret?
- Which single feature, if removed, would you expect to hurt the model most?

4.1 Define preprocessing constants

```
TARGET = 'Drafted'
ID_COL = 'Id'
HIGH_CARDINALITY_COL = 'School'

NUMERIC_MISSING_COLS = [
    'Age',
    'Sprint_40yd',
    'Vertical_Jump',
    'Bench_Press_Reps',
    'Broad_Jump',
    'Agility_3cone',
    'Shuttle',
]

CATEGORICAL_COLS = ['Player_Type', 'Position_Type', 'Position']
DOMAIN_FEATURES = ['SPEED_SCORE', 'EXPLOSIVENESS', 'AGILITY_RATIO']
```

4.2 Build the feature matrix

The helper rebuilds the same all-numeric matrix as Tutorials 1 and 2 — BMI, School_CE, missing indicators, the three domain features, and label-encoded categoricals. The target is never used to build a feature.

```
def prepare_features(train_raw, test_raw):
    train = train_raw.copy()
    test = test_raw.copy()

    # Simple features from Tutorial 1
    train['BMI'] = train['Weight'] / (train['Height'] ** 2)
    test['BMI'] = test['Weight'] / (test['Height'] ** 2)

    school_counts = train[HIGH_CARDINALITY_COL].value_counts()
    train['School_CE'] = train[HIGH_CARDINALITY_COL].map(school_counts).fillna(0)
    test['School_CE'] = test[HIGH_CARDINALITY_COL].map(school_counts).fillna(0)

    for c in NUMERIC_MISSING_COLS:
        train[f'{c}_missing'] = train[c].isna().astype(int)
        test[f'{c}_missing'] = test[c].isna().astype(int)

    # Mean imputation for numeric columns with missing values
    for c in NUMERIC_MISSING_COLS:
        mean_value = train[c].mean()
        train[c] = train[c].fillna(mean_value)
        test[c] = test[c].fillna(mean_value)

    # Domain features from Tutorial 2
    train['SPEED_SCORE'] = (train['Weight'] * 200.0) / (train['Sprint_40yd'] ** 4)
    test['SPEED_SCORE'] = (test['Weight'] * 200.0) / (test['Sprint_40yd'] ** 4)

    vj_mean = train['Vertical_Jump'].mean()
    vj_std = train['Vertical_Jump'].std()
    bj_mean = train['Broad_Jump'].mean()
    bj_std = train['Broad_Jump'].std()

    train['EXPLOSIVENESS'] = 0.5 * (
        (train['Vertical_Jump'] - vj_mean) / vj_std
        + (train['Broad_Jump'] - bj_mean) / bj_std
    )
    test['EXPLOSIVENESS'] = 0.5 * (
        (test['Vertical_Jump'] - vj_mean) / vj_std
        + (test['Broad_Jump'] - bj_mean) / bj_std
    )
```

```

)

train['AGILITY_RATIO'] = train['Agility_3cone'] / train['Shuttle'].where(train['Shuttle'] > 0, np.nan)
test['AGILITY_RATIO'] = test['Agility_3cone'] / test['Shuttle'].where(test['Shuttle'] > 0, np.nan)

ratio_median = train['AGILITY_RATIO'].median()
train['AGILITY_RATIO'] = train['AGILITY_RATIO'].fillna(ratio_median)
test['AGILITY_RATIO'] = test['AGILITY_RATIO'].fillna(ratio_median)

# Label encoding for low-cardinality categorical columns
for c in CATEGORICAL_COLS:
    categories = train[c].astype(str).unique()
    mapping = {cat: i for i, cat in enumerate(categories)}

    train[c] = train[c].astype(str).map(mapping).astype(int)
    test[c] = test[c].astype(str).map(mapping).fillna(-1).astype(int)

drop_cols = [ID_COL, HIGH_CARDINALITY_COL, TARGET]
feature_cols = [c for c in train.columns if c not in drop_cols]

X = train[feature_cols].copy()
y = train[TARGET].astype(int)
X_test = test[feature_cols].copy()

return X, y, X_test, feature_cols

```

```
X, y, X_test, feature_cols = prepare_features(train_raw, test_raw)
```

```
print(f'Number of features: {len(feature_cols)}')
print(feature_cols)
```

```
Number of features: 25
['Year', 'Age', 'Height', 'Weight', 'Sprint_40yd', 'Vertical_Jump', 'Bench_Press_Reps', 'Broad_Jump', 'Agility_3cone', 'Shuttle']
```

✓ 5. CatBoost baseline

Before tuning anything, we score a default CatBoost with the same cross-validation. Without this reference point a tuned score means nothing — we could not tell whether the search helped. CatBoost is a gradient-boosting model: it builds trees in sequence, each correcting the last, and tends to do well on tabular data.

✓ 5.1 Import CatBoost and define CV

```

from sklearn.model_selection import StratifiedKFold
from sklearn.metrics import roc_auc_score
from catboost import CatBoostClassifier
import optuna

optuna.logging.set_verbosity(optuna.logging.WARNING)

skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)

```

✓ 5.2 Cross-validation helper for CatBoost

Same 5-fold OOF setup as before, with one addition: **early stopping**, which halts training once the validation AUC stops improving. CV matters even more here — a single split would make each tuning trial depend too much on luck.

```

def catboost_cv(params, X, y, X_test=None):
    oof = np.zeros(len(X))
    test_pred = np.zeros(len(X_test)) if X_test is not None else None
    fold_aucs = []

    for fold, (tr_idx, va_idx) in enumerate(skf.split(X, y), start=1):
        X_tr, X_va = X.iloc[tr_idx].copy(), X.iloc[va_idx].copy()
        y_tr, y_va = y.iloc[tr_idx], y.iloc[va_idx]

        model = CatBoostClassifier(**params)
        model.fit(
            X_tr,
            y_tr,

```

```

        eval_set=(X_va, y_va),
        verbose=False,
    )

    va_pred = model.predict_proba(X_va)[: , 1]
    oof[va_idx] = va_pred

    fold_auc = roc_auc_score(y_va, va_pred)
    fold_aucs.append(fold_auc)

    if X_test is not None:
        test_pred += model.predict_proba(X_test)[: , 1] / skf.n_splits

    print(f'Fold {fold}: AUC = {fold_auc:.4f}')

oof_auc = roc_auc_score(y, oof)
print(f'OOF AUC: {oof_auc:.4f}')

return oof, test_pred, oof_auc, fold_aucs

```

5.3 Evaluate the default CatBoost

```

default_params = {
    'loss_function': 'Logloss',
    'eval_metric': 'AUC',
    'iterations': 1000,
    'random_seed': SEED,
    'verbose': False,
    'allow_writing_files': False,
}

default_oof, default_test_pred, default_auc, default_fold_aucs = catboost_cv(
    default_params,
    X,
    y,
    X_test,
)

print(f'Default CatBoost OOF AUC: {default_auc:.4f}')

Fold 1: AUC = 0.8144
Fold 2: AUC = 0.8614
Fold 3: AUC = 0.8593
Fold 4: AUC = 0.8111
Fold 5: AUC = 0.8470
OOF AUC: 0.8327
Default CatBoost OOF AUC: 0.8327

```

Questions to think about:

- Why establish a default-model score *before* tuning at all?
- The default runs 1000 iterations with no early stopping. Could it be over- or under-fitting?

6. Hyperparameter tuning with Optuna

CatBoost has too many parameters to try every combination. Optuna searches efficiently — it uses earlier trials to decide what to try next instead of sweeping a fixed grid. It is not magic, though: the result still depends on the search space, the number of trials, and whether the winning settings generalize past this CV split.

6.1 Fast mode

Hyperparameter tuning can take time because each trial runs cross-validation.

For teaching and debugging, `FAST_MODE=True` keeps the notebook reasonably quick. For a more serious run, set `FAST_MODE=False`.

```

# Set FAST_MODE = True for a quick run.
# Set FAST_MODE = False when you want to run a more serious search.
FAST_MODE = True

```

```

N_TRIALS = 10 if FAST_MODE else 60
TUNING_ITERATIONS = 600 if FAST_MODE else 1500

print('FAST_MODE:', FAST_MODE)
print('N_TRIALS:', N_TRIALS)
print('TUNING_ITERATIONS:', TUNING_ITERATIONS)

```

```

FAST_MODE: True
N_TRIALS: 10
TUNING_ITERATIONS: 600

```

6.2 Define the search space

The search space decides *what* Optuna may change — arguably the most important choice in tuning. Too narrow and it cannot find anything better; too wide and it wastes trials on unstable settings. Here we let it vary the parameters controlling tree complexity, learning speed, regularization, randomness, and sampling. The point is not to memorize each knob but to recognize which behavior each group controls.

```

fixed_params = {
    'loss_function': 'Logloss',
    'eval_metric': 'AUC',
    'iterations': TUNING_ITERATIONS,
    'early_stopping_rounds': 100,
    'use_best_model': True,
    'random_seed': SEED,
    'verbose': False,
    'allow_writing_files': False,
}

def objective(trial):
    bootstrap_type = trial.suggest_categorical(
        'bootstrap_type',
        ['Bayesian', 'Bernoulli', 'MVS'],
    )

    params = dict(
        fixed_params,
        depth=trial.suggest_int('depth', 4, 10),
        learning_rate=trial.suggest_float('learning_rate', 1e-3, 0.3, log=True),
        l2_leaf_reg=trial.suggest_float('l2_leaf_reg', 1e-2, 1e2, log=True),
        random_strength=trial.suggest_float('random_strength', 0.0, 2.0),
        rsm=trial.suggest_float('rsm', 0.5, 1.0),
        border_count=trial.suggest_int('border_count', 32, 255),
        leaf_estimation_iterations=trial.suggest_int('leaf_estimation_iterations', 1, 10),
        bootstrap_type=bootstrap_type,
    )

    if bootstrap_type == 'Bayesian':
        params['bagging_temperature'] = trial.suggest_float('bagging_temperature', 0.0, 10.0)
    elif bootstrap_type == 'Bernoulli':
        params['subsample'] = trial.suggest_float('subsample', 0.5, 1.0)

    _, _, auc, _ = catboost_cv(params, X, y, X_test=None)
    return auc

study = optuna.create_study(
    direction='maximize',
    sampler=optuna.samplers.TPESampler(seed=SEED),
)

study.optimize(objective, n_trials=N_TRIALS, show_progress_bar=True)

print(f'Best OOF AUC: {study.best_value:.4f}')
print('Best params:')
print(study.best_params)

```

Best trial: 6. Best value: 0.827639: 100%

10/10 [01:13<00:00, 6.38s/it]

Fold 1: AUC = 0.8071
 Fold 2: AUC = 0.8585
 Fold 3: AUC = 0.8542
 Fold 4: AUC = 0.7902
 Fold 5: AUC = 0.8433
 OOF AUC: 0.7704
 Fold 1: AUC = 0.8060
 Fold 2: AUC = 0.8480
 Fold 3: AUC = 0.8571
 Fold 4: AUC = 0.7838
 Fold 5: AUC = 0.8455
 OOF AUC: 0.7521
 Fold 1: AUC = 0.8050
 Fold 2: AUC = 0.8617
 Fold 3: AUC = 0.8568
 Fold 4: AUC = 0.7858
 Fold 5: AUC = 0.8422
 OOF AUC: 0.7641
 Fold 1: AUC = 0.8104
 Fold 2: AUC = 0.8596
 Fold 3: AUC = 0.8563
 Fold 4: AUC = 0.7825
 Fold 5: AUC = 0.8393
 OOF AUC: 0.7736
 Fold 1: AUC = 0.8111
 Fold 2: AUC = 0.8663
 Fold 3: AUC = 0.8608
 Fold 4: AUC = 0.7890
 Fold 5: AUC = 0.8459
 OOF AUC: 0.7686
 Fold 1: AUC = 0.8002
 Fold 2: AUC = 0.8509
 Fold 3: AUC = 0.8489
 Fold 4: AUC = 0.7798
 Fold 5: AUC = 0.8446
 OOF AUC: 0.7816
 Fold 1: AUC = 0.8098
 Fold 2: AUC = 0.8505
 Fold 3: AUC = 0.8607
 Fold 4: AUC = 0.8021
 Fold 5: AUC = 0.8392
 OOF AUC: 0.8276
 Fold 1: AUC = 0.8124
 Fold 2: AUC = 0.8599
 Fold 3: AUC = 0.8493
 Fold 4: AUC = 0.7889
 Fold 5: AUC = 0.8371
 OOF AUC: 0.7647
 Fold 1: AUC = 0.8111
 Fold 2: AUC = 0.8414
 Fold 3: AUC = 0.8310
 Fold 4: AUC = 0.7911
 Fold 5: AUC = 0.8411
 OOF AUC: 0.8219
 Fold 1: AUC = 0.7994
 Fold 2: AUC = 0.8357
 Fold 3: AUC = 0.8481
 Fold 4: AUC = 0.8000
 Fold 5: AUC = 0.8555
 OOF AUC: 0.7455
 Best OOF AUC: 0.8276
 Best params:

```
{'bootstrap_type': 'MVS', 'depth': 4, 'learning_rate': 0.09708210658008556, 'l2_leaf_reg': 0.01987021538542863, 'random_strength'
```

6.3 Inspect the Optuna results

We look at the best score and parameters, the optimization history (still improving, or flat?), and a rough parameter-importance plot. These are diagnostics — the real verdict is the tuned model's CV score against the default.

Questions to think about:

- Did the optimization history flatten out, or would more trials likely help?
- Which parameters mattered most — and if any best value hit the edge of its range, should you widen it?
- On this small dataset a single trial can spike to a high AUC. Is that real signal or CV variance?

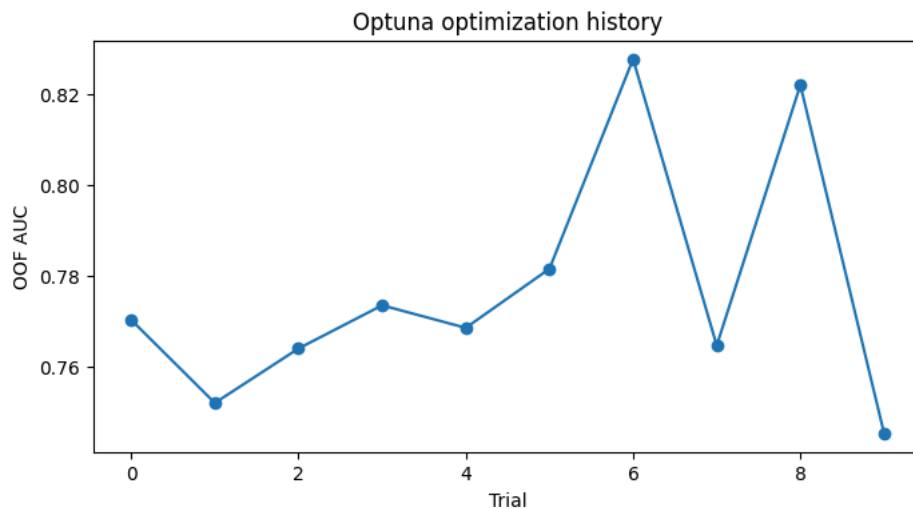
```

trials_df = study.trials_dataframe()
completed_trials = trials_df[trials_df['state'] == 'COMPLETE'].copy()

plt.figure(figsize=(8, 4))
plt.plot(completed_trials['number'], completed_trials['value'], marker='o')
plt.xlabel('Trial')
plt.ylabel('OOB AUC')
plt.title('Optuna optimization history')
plt.show()

completed_trials[['number', 'value']].sort_values('value', ascending=False).head(10)

```



number	value	
6	0.827639	
8	0.821944	
5	0.781588	
3	0.773569	
0	0.770386	
4	0.768622	
7	0.764745	
2	0.764085	
1	0.752130	
9	0.745506	

Parameter importance is an optional diagnostic. It gives a rough idea of which hyperparameters were influential in this search.

```

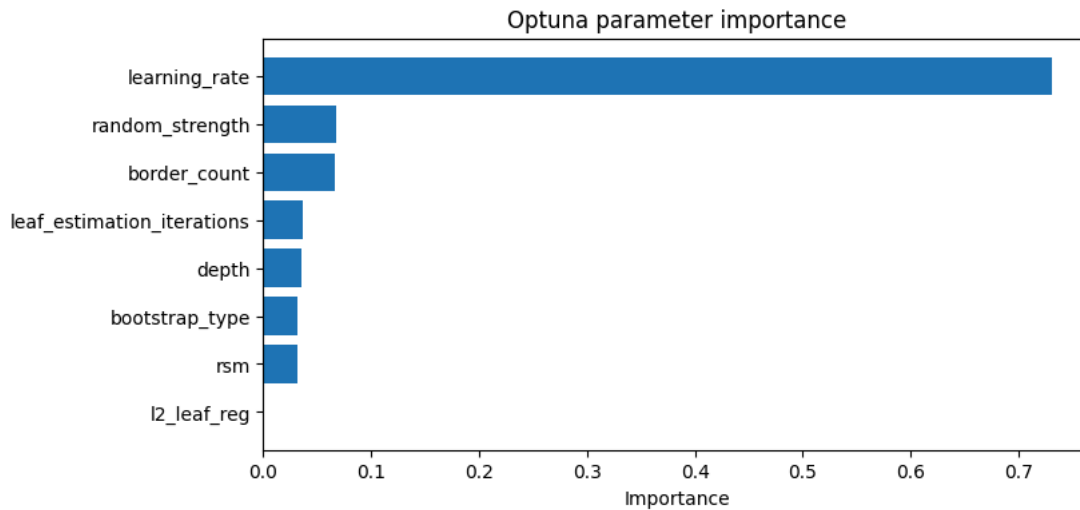
try:
    from optuna.importance import get_param_importances

    param_importance = get_param_importances(study)
    param_importance = pd.DataFrame({
        'parameter': list(param_importance.keys()),
        'importance': list(param_importance.values()),
    })
    display(param_importance)

    plt.figure(figsize=(8, 4))
    plot_df = param_importance.sort_values('importance', ascending=True)
    plt.barh(plot_df['parameter'], plot_df['importance'])
    plt.title('Optuna parameter importance')
    plt.xlabel('Importance')
    plt.show()
except Exception as e:
    print('Parameter importance could not be calculated:', e)

```

	parameter	importance	
0	learning_rate	0.731002	
1	random_strength	0.067383	
2	border_count	0.066380	
3	leaf_estimation_iterations	0.036275	
4	depth	0.035449	
5	bootstrap_type	0.031713	
6	rsm	0.031120	
7	l2_leaf_reg	0.000679	



Next steps: [Generate code with param_importance](#) [New interactive sheet](#)

7. Refit the tuned model

After Optuna finds the best parameters, we evaluate the tuned CatBoost again with 5-fold CV and create fold-averaged test predictions.

We then compare the tuned model with the default model. The tuned model is not guaranteed to win, especially in `FAST_MODE`. Validation results should decide which prediction we use.

```
best_params = dict(fixed_params, **study.best_params)

tuned_oof, tuned_test_pred, tuned_auc, tuned_fold_aucs = catboost_cv(
    best_params,
    X,
    y,
    X_test,
)

score_table = pd.DataFrame([
    {
        'model': 'Default CatBoost',
        'OOF AUC': default_auc,
        'fold_auc_mean': np.mean(default_fold_aucs),
        'fold_auc_std': np.std(default_fold_aucs),
    },
    {
        'model': 'Tuned CatBoost',
        'OOF AUC': tuned_auc,
        'fold_auc_mean': np.mean(tuned_fold_aucs),
        'fold_auc_std': np.std(tuned_fold_aucs),
    },
]).sort_values('OOF AUC', ascending=False)

score_table.style.format({
    'OOF AUC': '{:.4f}',
    'fold_auc_mean': '{:.4f}',
```

```
'fold_auc_std': '{:.4f}',
})
```

```
Fold 1: AUC = 0.8098
Fold 2: AUC = 0.8505
Fold 3: AUC = 0.8607
Fold 4: AUC = 0.8021
Fold 5: AUC = 0.8392
OOB AUC: 0.8276
```

	model	OOB AUC	fold_auc_mean	fold_auc_std
0	Default CatBoost	0.8327	0.8387	0.0217
1	Tuned CatBoost	0.8276	0.8325	0.0228

```
if tuned_auc >= default_auc:
    selected_method = 'Tuned CatBoost'
    test_pred = tuned_test_pred
else:
    selected_method = 'Default CatBoost'
    test_pred = default_test_pred

print('Selected submission method:', selected_method)
print('test_pred shape:', test_pred.shape)
```

```
Selected submission method: Default CatBoost
test_pred shape: (696,)
```

Questions to think about:

- In `FAST_MODE` the tuned model is **not** guaranteed to beat the default. Does it here, and is the gap bigger than `fold_auc_std`?
- Would averaging several tuned models (different seeds) give a more trustworthy winner than one study?

8. Create the submission file

We use the selected CatBoost prediction and create `submission.csv`.

Saving the prediction as a CSV file [DO NOT CHANGE]

WARNING: Do not change the following cell unless you understand what it is doing.

The final CSV must exactly match the expected format. By using `sample_submission.csv`, we preserve the required row order and column structure. Here, we replace only the `Drafted` column with our predictions.

```
submission = pd.read_csv(sample_sub_file)
submission['Drafted'] = test_pred

out_dir = WORK_DIR / 'output'
out_dir.mkdir(exist_ok=True)

submission_path = out_dir / 'submission.csv'
submission.to_csv(submission_path, index=False)

print('Saved to:', submission_path)
submission.head()
```

Saved to: /content/drive/MyDrive/competition/output/submission.csv

	Id	Drafted
0	2781	0.592125
1	2782	0.882609
2	2783	0.950868
3	2784	0.927150
4	2785	0.817886

Next steps: [Generate code with submission](#) [New interactive sheet](#)

9. Wrap-up

In this notebook we held the features fixed, scored a default model first, then let an automatic search propose better settings — and compared the two with the *same* cross-validation.

A tuned model is a claim. Cross-validation against the default is how you check the claim.

Questions to think about:

- **Tutorial 4** combines several models. Would your tuned CatBoost add diversity to an ensemble, or just duplicate another model?
- Re-run with `FAST_MODE = False` and more trials — does the conclusion change?

*If a cell raises an error, re-run everything from **Section 2** so the raw data is rebuilt.*